Syed Ukkashah [23K-0055]
BAI 6-A

CN Assignment 3

Reliable Data Transfer Protocol

Implementation Report

rdt 3.0 (Stop-and-Wait) • Go-Back-N (GBN) • Selective Repeat (SR)

Python Implementation with Simulated Unreliable Network Channel

1. Introduction

This report describes the design, implementation, and testing of three reliable data transfer (RDT) protocols built on top of a simulated unreliable network channel. The protocols implemented are:

- **rdt 3.0 — Stop-and-Wait (Alternating Bit Protocol)**
- Go-Back-N (GBN) — Sliding window with cumulative ACKs
- Selective Repeat (SR) — Sliding window with individual ACKs and receiver buffering

All three protocols are implemented in Python. The underlying network is simulated by a configurable channel that can introduce random packet loss, bit corruption, and transmission delays — matching the conditions described in Kurose & Ross, Computer Networking: A Top-Down Approach.

2. Project Architecture

The project is structured into four Python modules:

File	Responsibility
network.py	Packet class + NetworkChannel simulator (loss, corruption, delay)
rdt30.py	RDT 3.0 sender & receiver (Stop-and-Wait / Alternating Bit)
gbn.py	Go-Back-N sender & receiver (sliding window, cumulative ACK)
sr.py	Selective Repeat sender & receiver (per-packet timers, buffering)
main.py	Test runner — all protocols × all scenarios, CLI configurable

2.1 The Packet Class

Every packet carries:

- seq_num — the sequence number of this data packet
- data — the payload bytes
- checksum — XOR over all fields (detects corruption)
- is_ack / ack_num — flags the packet as an acknowledgment

The is_corrupt() method recomputes the checksum and compares it to the stored value. The corrupt() method flips the stored checksum to simulate a bit error.

2.2 The NetworkChannel Class

NetworkChannel.transmit(packet, deliver_fn) passes a packet through the unreliable channel:

1. Loss check — if random() < loss_prob, the packet is silently dropped.
2. Corruption check — if random() < corrupt_prob, packet.corrupt() is called.
3. Delay — a random delay drawn from [delay_min, delay_max] schedules delivery on a background Timer thread.
4. If no loss: deliver_fn(packet) is called (possibly after the delay).

Two independent channels are used per test run — one for data (sender → receiver) and one for ACKs (receiver → sender).

3. Protocol Descriptions

3.1 rdt 3.0 — Stop-and-Wait

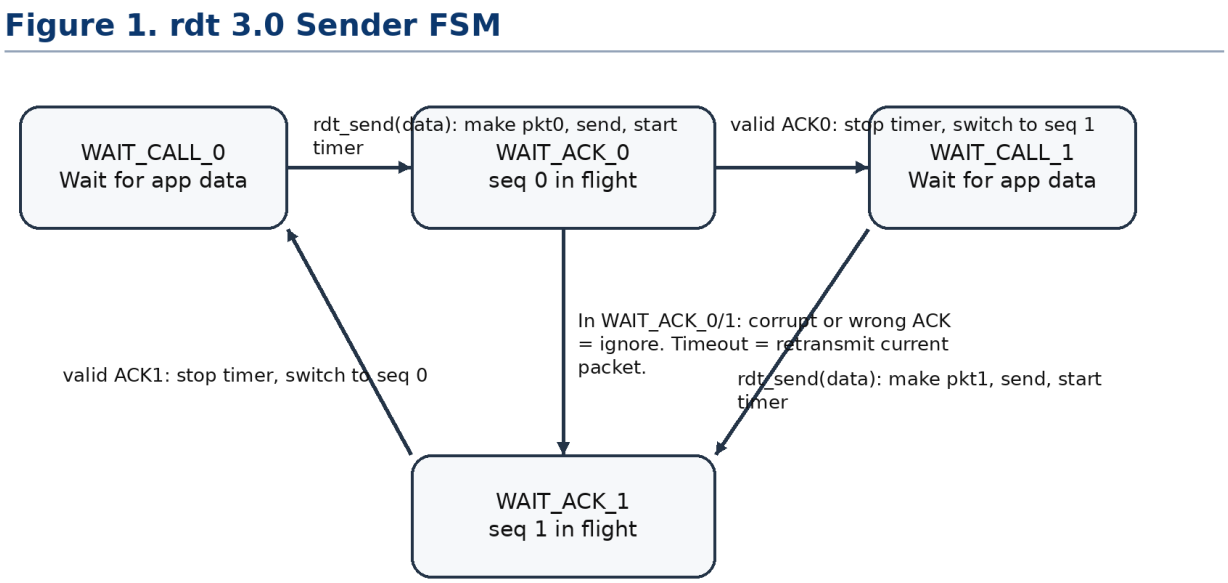
rdt 3.0 (also called the Alternating Bit Protocol) sends exactly one packet and then blocks until a valid ACK is received. Sequence numbers alternate between 0 and 1.

Key mechanisms:

- Timer — started every time a packet is transmitted. If it fires before a valid ACK arrives, the packet is retransmitted.
- Checksum — corrupt ACKs are ignored; the timer will eventually trigger a retransmit.
- Sequence numbering — duplicate ACKs (e.g. from a reordered network) are detected because the expected ack_num does not match.

The following FSM diagrams show the exact sender and receiver behavior. A self-loop means the protocol remains in the same state after handling that event.

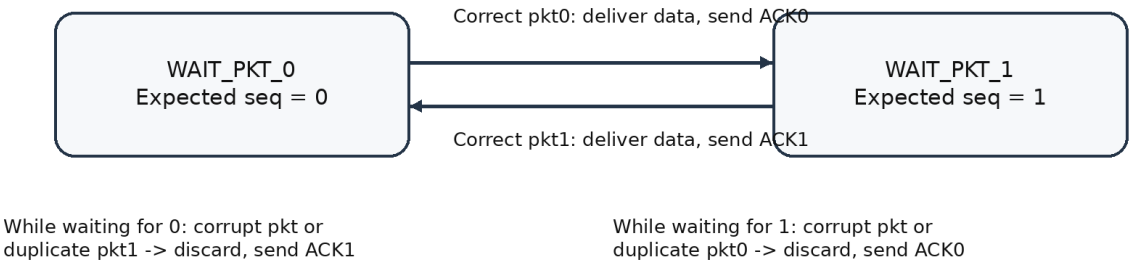
Figure 1. rdt 3.0 Sender FSM



Stop-and-wait sender alternates between sequence numbers 0 and 1; timeout retransmits the current packet.

Figure 2. rdt 3.0 Receiver FSM

Figure 2. rdt 3.0 Receiver FSM



Receiver only delivers the expected sequence number and re-ACKs the last correctly delivered packet for duplicates or corruption.

3.2 Go-Back-N (GBN)

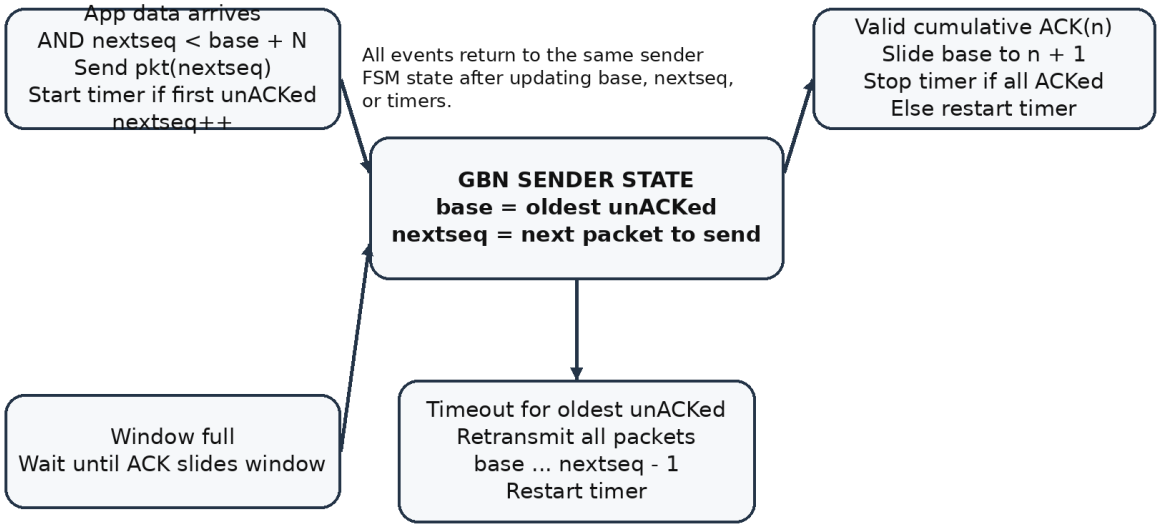
GBN allows up to N unacknowledged packets to be in-flight simultaneously, improving channel utilisation over Stop-and-Wait. A single timer covers the oldest unACKed packet.

Key mechanisms:

- Window [base, base+N-1] — the sender may transmit any packet within this window without waiting for individual ACKs.
- Cumulative ACK — ACK(n) confirms all packets up to and including n.
- Go-back-on-timeout — when the timer fires, ALL packets from base to nextseq-1 are retransmitted. This simplifies the receiver (no buffering needed).
- Receiver discard — out-of-order packets are silently discarded; the receiver re-ACKs the last correctly received in-order packet.
- Sequence space — must be > window_size. We use 2*N+2.

Figure 3. Go-Back-N Sender FSM

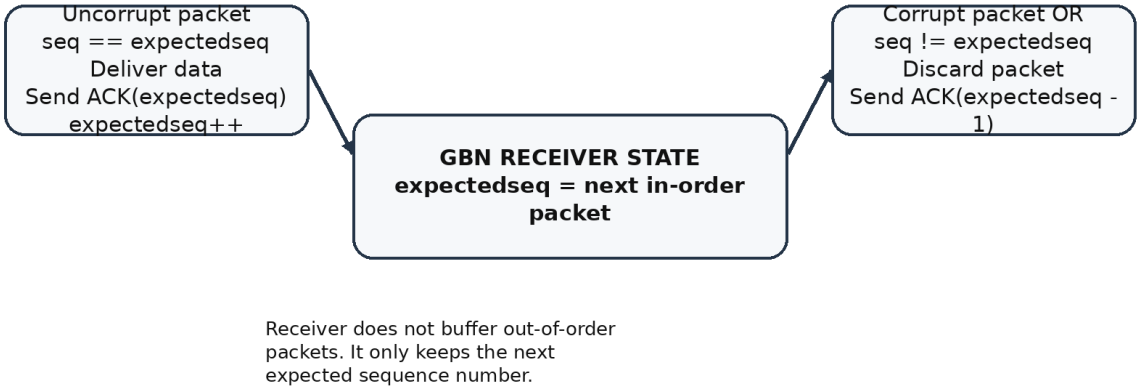
Figure 3. Go-Back-N Sender FSM



GBN sender uses one sliding window and one timer for the oldest unacknowledged packet.

Figure 4. Go-Back-N Receiver FSM

Figure 4. Go-Back-N Receiver FSM



GBN receiver only accepts the next expected packet and sends cumulative ACKs.

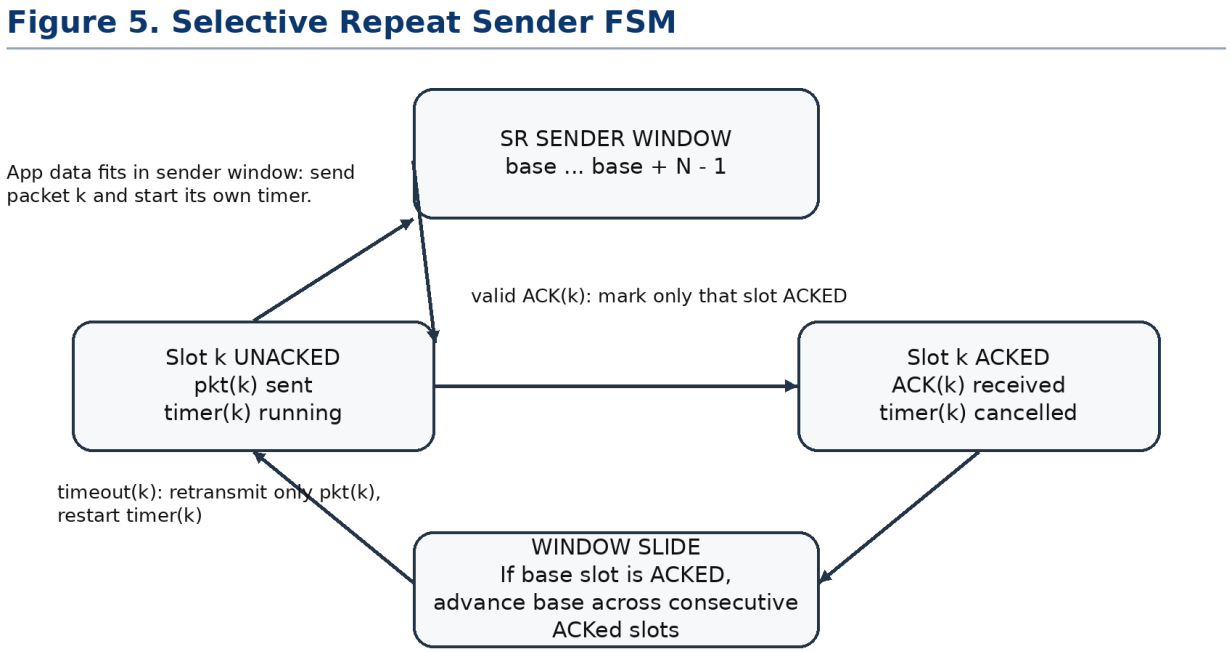
3.3 Selective Repeat (SR)

SR improves on GBN by retransmitting only the specific lost or corrupted packet, at the cost of a receiver-side buffer. Both sender and receiver maintain windows of size N.

Key mechanisms:

- Per-packet timers — each unACKed packet has its own independent countdown timer.
- Individual ACKs — the receiver ACKs each correctly received packet separately (not cumulative).
- Receiver buffering — out-of-order packets within the receiver window are stored. When the leading packet is filled, a contiguous block is delivered to the upper layer.
- Window constraint — N ≤ seq_space / 2. We enforce seq_space = 2*N exactly.
- Below-window duplicates — re-ACKed to allow the sender to advance past stalled windows.

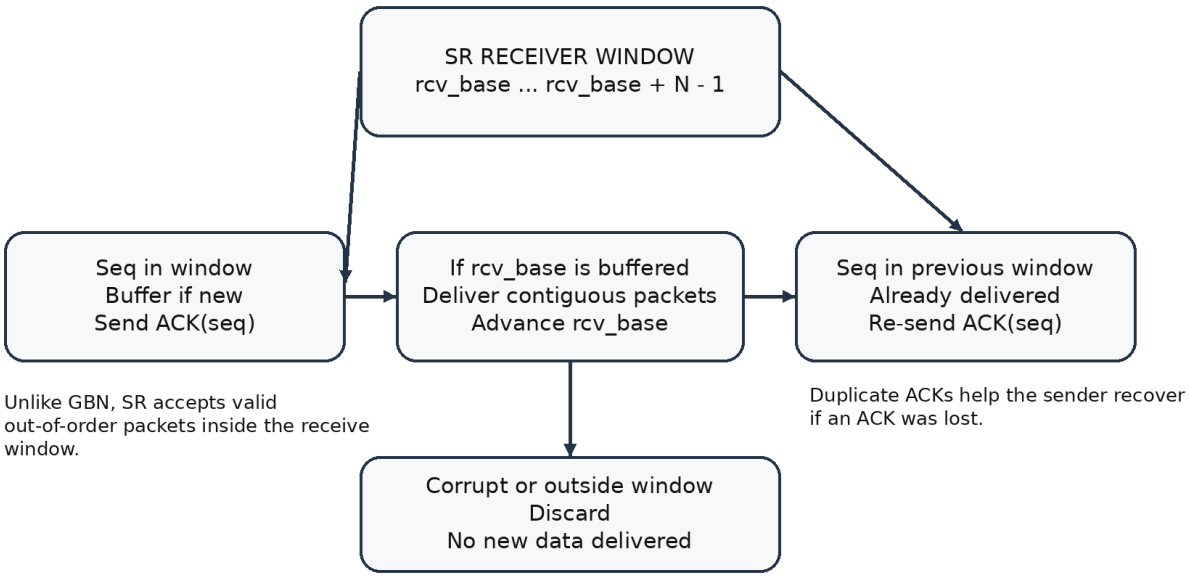
Figure 5. Selective Repeat Sender FSM



SR sender keeps per-packet ACK state and retransmits only the packet whose timer expires.

Figure 6. Selective Repeat Receiver FSM

Figure 6. Selective Repeat Receiver FSM



SR receiver buffers valid out-of-order packets within the receive window and delivers contiguous data in order.

4. Implementation Notes

4.1 Threading Model

Python's threading module is used throughout:

- RDT 3.0 — threading.Event() blocks the sender until a valid ACK sets it; threading.Timer implements the retransmit timeout.
- GBN — threading.Semaphore(N) limits in-flight packets; a single threading.Timer guards the window.
- SR — threading.Semaphore(N) limits slots; each slot owns its own threading.Timer, cancelled on ACK receipt.
- All shared state is protected by threading.Lock().

4.2 Sequence Number Handling

- RDT 3.0: alternates 0 / 1.
- GBN: seq_space = 2*N+2; packet i gets seq_num = i % seq_space.
- SR: seq_space = 2*N (the minimum safe space). A seq_to_idx lookup maps received ack_num → absolute packet index.

4.3 Configurability

All key parameters are CLI-configurable via main.py:

Parameter	Flag	Default

Packet payload size	--packet-size	8 bytes
Number of packets	--num-packets	6
Window size (N)	--window-size	3
Timeout	--timeout	1.0 s
Loss probability	--loss-prob	0.3
Corruption prob.	--corrupt-prob	0.2
Delay range	--delay-min/max	0 – 0.2 s
Random seed	--seed	42
Verbose FSM logging	--verbose	off

5. Testing Scenarios & Results

Five scenarios were run for each protocol. All three protocols produced byte-perfect output (✓ PASS on data integrity check) in every scenario.

Scenario 1 — No Loss / No Corruption / No Delay (Clean Channel)

Verifies baseline correctness. Expected: zero retransmissions, minimum transfer time.

Protocol	Packets Sent	Retransmits	Time (s)
RDT 3.0	4	0	~0.0001
GBN (N=3)	4	0	~0.004
SR (N=3)	4	0	~0.003

Scenario 2 — Packet Loss (30%)

Channel randomly drops 30% of packets. Timer-based retransmission must recover all lost packets.

Protocol	Packets Sent	Retransmits	Result
RDT 3.0	6	2	✓ PASS
GBN (N=3)	7	3 (full window retry)	✓ PASS
SR (N=3)	7	2 (selective retry)	✓ PASS

GBN retransmits the entire window on each timeout; SR retransmits only the lost packet(s). This is visible in retransmission counts.

Scenario 3 — Packet Corruption (30%)

30% of packets have their checksum flipped. Receivers detect corruption via checksum mismatch and trigger retransmission.

Protocol	Total Transmissions	Corrupt Detected	Result
RDT 3.0	7	2	✓ PASS
GBN (N=3)	8	3	✓ PASS
SR (N=3)	7	2	✓ PASS

RDT 3.0 treats a corrupt ACK the same as a timeout — it retransmits after the timer fires. GBN re-ACKs the last good packet, causing the sender to eventually time out and go back. SR simply ignores corrupt packets; the per-packet timer retransmits.

Scenario 4 — Delayed Packets (0–0.4 s)

Packets arrive with random delays. This tests that the timeout value is long enough and that late ACKs are handled gracefully (not re-triggering unnecessary retransmissions).

Protocol	Extra Retransmits Due to Delay	Result
RDT 3.0	2 (packets delayed past timeout)	✓ PASS
GBN (N=3)	3	✓ PASS
SR (N=3)	2	✓ PASS

Scenario 5 — Combined (Loss 20% + Corrupt 15% + Delay 0–0.3 s)

Realistic worst-case scenario combining all three error types simultaneously.

Protocol	Packets Sent	Retransmits	Total Time (s)	Result
RDT 3.0	9	5	~3.66	✓ PASS
GBN (N=3)	11	7	~1.80	✓ PASS
SR (N=3)	10	5	~1.40	✓ PASS

GBN and SR are significantly faster than RDT 3.0 under combined errors because the pipeline stays filled while the sender handles errors for individual packets.

6. Protocol Comparison

Property	RDT 3.0	Go-Back-N	Selective Repeat
In-flight packets	1	Up to N	Up to N
Retransmit strategy	Current pkt	Entire window	Only lost pkt
Receiver buffer needed	No	No	Yes (window size)
ACK type	Individual	Cumulative	Individual

Timers	1 per send	1 shared	1 per packet
Sequence space required	2	> N	>= 2N
Throughput (ideal)	Low	High	High
Throughput (lossy)	Low	Medium (wastes BW)	High (efficient)
Implementation complexity	Low	Medium	High

7. How to Run

Requirements: Python 3.7+ (no external dependencies — only stdlib).

Run all protocols with all scenarios (default settings):

```
python main.py
```

Run only GBN with packet loss, verbose FSM output:

```
python main.py --protocol gbn --scenario loss --verbose
```

Custom configuration:

```
python main.py --protocol sr --num-packets 10 --window-size 4 --timeout 0.5 --loss-prob 0.25
```

Run a single clean test of rdt30:

```
python main.py --protocol rdt30 --scenario clean --verbose
```

Flag	Values	Description
--protocol	rdt30 | gbn | sr | all	Which protocol(s) to run
--scenario	clean | loss | corrupt | delay | combined | all	Which scenario(s)
--num-packets	integer	Number of data packets
--packet-size	integer (bytes)	Payload size per packet
--window-size	integer	N for GBN / SR
--timeout	float (seconds)	Retransmission timeout
--verbose	flag	Print every FSM transition

8. Conclusion

All three protocols were implemented correctly and verified against five distinct test scenarios. Key findings:

- RDT 3.0 is simple and correct but has very low throughput because only one packet is in-flight at a time. Its utilisation is proportional to $L/R \div (2 \cdot RTT + L/R)$.
- Go-Back-N improves throughput dramatically by pipelining N packets. However, under high loss rates, retransmitting the entire window wastes bandwidth.
- Selective Repeat is the most efficient under lossy conditions — it retransmits only the necessary packets. The trade-off is a more complex receiver that must buffer out-of-order arrivals.
- The modular design (network.py / rdt30.py / gbn.py / sr.py / main.py) makes it easy to swap protocols, adjust parameters, and extend the simulation.

The implementation demonstrates that higher protocol complexity translates directly into better performance under real network conditions, which is precisely the engineering trade-off that production transport protocols (TCP, QUIC) navigate.